



Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

Sistemas de Computación
Trabajo Práctico N° 5

Docentes:
Ing. Miguel Ángel Solinas
Ing. Javier Alejandro Jorge

Autores:
Matías Lucas Morales Robles
José Ignacio Torres
Esteban Samuel Suarez

Índice

1. Introducción	2
2. El concepto de driver	3
3. Tipos de drivers	3
4. El concepto de Character Device Driver (CDD)	4
4.1. Modelo de capas	4
4.2. Números mayor y minor	4
5. Entorno de desarrollo	5
5.1. Verificando el entorno	5
5.2. Instalando dependencias	5
6. Ejemplos de drivers propuestos por la cátedra	5
6.1. Módulo básico (drv1)	5
6.2. Registro de mayor/minor (drv2)	7
6.3. CDD completo usando /dev (drv3)	8
6.4. R/W con espacio de usuario (drv4)	9
7. Generación de señales	10
7.1. Arquitectura usada para la comunicación	10
7.2. Señal PDM	10
7.3. Orden de arranque	11
8. CDD	12
8.1. Flujo del sistema	12
8.2. Inicialización y Lectura de Descriptores GPIO	13
8.3. Registro del Dispositivo y Operaciones de Archivo	14
8.4. Justificación Arquitectónica	14
8.5. Validación Funcional	14
9. Aplicación de usuario en Python	14
9.1. Flujo de Ejecución de la Aplicación	14
9.2. Detalles de Implementación	16
9.3. Acoplamiento con el Driver	16
10. Prueba final	18
10.1. Terminal 1: Generación de señales	18
10.2. Terminal 2: QEMU	18
10.3. Terminal 3: SSH en la Raspberry Pi simulada	18
11. Conclusión	20

1. Introducción

En este trabajo práctico se aborda el diseño y la programación de módulos del kernel de Linux, con especial énfasis en la implementación de un controlador de dispositivo de caracteres (*Character Device Driver* o CDD).

El objetivo principal consiste en desarrollar un *driver* capaz de muestrear dos señales GPIO externas con una periodicidad de un segundo, complementado por una aplicación en el espacio de usuario encargada de adquirir una de estas señales y graficarla en tiempo real.

Para la validación del sistema, el entorno de desarrollo se desplegó sobre una plataforma Raspberry Pi virtualizada mediante QEMU. La solución implementada consta de un módulo del kernel que expone el nodo de dispositivo `/dev/sensor_driver`, permitiendo que una aplicación en Python interactúe con dicho recurso desde el espacio de usuario de manera eficiente.

2. El concepto de driver

Un *driver* es un software que permite al sistema operativo interactuar con un periférico, abstrayendo el hardware y proporcionando una interfaz estandarizada para su utilización.

En este contexto, se distinguen dos conceptos relacionados:

- **Device driver:** Pieza de software que controla un dispositivo físico.
- **Device controller:** Periférico que controla otros dispositivos y que, a su vez, requiere de su propio *driver* (denominado *bus driver*).

Todo *device driver* consta de dos partes diferenciadas:

1. Una parte específica del dispositivo (asociada a qué hace el hardware).
2. Una parte específica del sistema operativo (encargada de cómo se comunica con el *kernel*).

3. Tipos de drivers

Basándose en la interfaz específica del sistema operativo, Linux clasifica los *drivers* en tres categorías verticales según su naturaleza de acceso y abstracción:

Tipo	Orientación	Ejemplos
Network	Paquetes	Ethernet, Wi-Fi, Bluetooth, Módems LTE
Block	Bloques	Discos rígidos (HDD), SSDs, dispositivos NVMe, tarjetas SD
Character	Bytes	Puertos seriales, placas de audio, cámaras de video, GPIO, teclados

Por otro lado, el directorio `/dev` aloja los archivos de dispositivo especiales (*device nodes*) que representan a todos los periféricos del sistema. En particular, los *character devices* activos en el sistema y sus nodos correspondientes pueden ser consultados y listados mediante los siguientes comandos:

```
ls -l /dev/ | grep "^c"
cat /proc/devices
```

4. El concepto de Character Device Driver (CDD)

El *Character Device Driver* (CDD) constituye la categoría de controlador más común en el sistema operativo Linux. De este modo, cualquier *driver* que no se clasifique dentro de las funciones de almacenamiento ni de red es considerado, en alguna de sus variantes, un CDD.

4.1. Modelo de capas

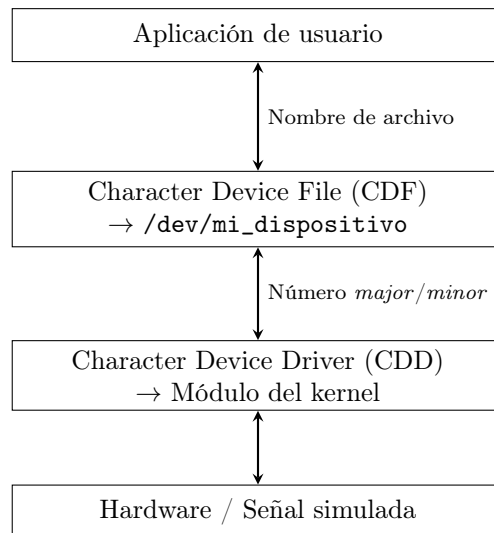


Figura 1: Modelo de capas del sistema.

Este esquema ilustra el modelo de capas del sistema. El vínculo entre la aplicación y el CDF se establece a través del nombre del archivo. Por el contrario, la relación entre el CDF y el CDD se basa estrictamente en el número *major* y *minor*, de forma independiente al nombre del dispositivo.

4.2. Números major y minor

```
# Ver dispositivos de caracteres registrados
cat /proc/devices
```

```
# Ejemplo de salida:
# 254 gpiochip
```

En este marco, cada dispositivo se encuentra asociado a dos identificadores fundamentales:

- **Major number:** Identifica el *driver* principal.
- **Minor number:** Identifica el dispositivo específico gestionado por dicho *driver*.

A nivel del *kernel*, la estructura de datos que contiene ambos identificadores es el tipo `dev_t`, el cual posee una longitud de 32 bits (12 bits reservados para el *major* y 20 bits para el *minor*). Para la manipulación de estos identificadores, el sistema provee las siguientes macros de utilidad:

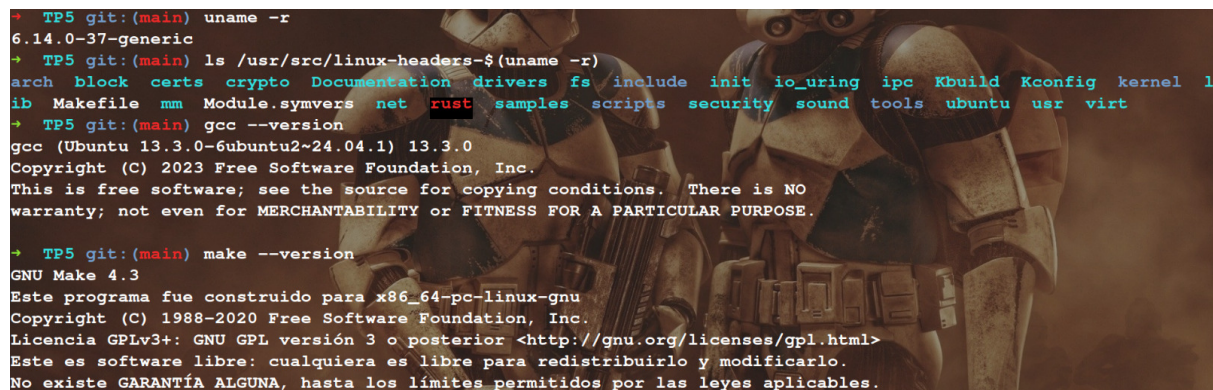
```
MAJOR(dev_t dev)           // extrae el major
MINOR(dev_t dev)           // extrae el minor
MKDEV(int major, int minor) // construye un dev_t
```

5. Entorno de desarrollo

5.1. Verificando el entorno

A modo de verificación del entorno, se deben ejecutar los siguientes comandos para comprobar las herramientas de compilación y los recursos del sistema:

```
uname -r                # versión del kernel
ls /usr/src/linux-headers-$(uname -r)  # headers instalados
gcc --version
make --version
```



```
+ TP5 git:(main) uname -r
6.14.0-37-generic
+ TP5 git:(main) ls /usr/src/linux-headers-$(uname -r)
arch block certs crypto Documentation drivers fs include init io_uring ipc Kbuild Kconfig kernel lib
ib Makefile mm Module.symvers net rust samples scripts security sound tools ubuntu usr virt
+ TP5 git:(main) gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
+ TP5 git:(main) make --version
GNU Make 4.3
Este programa fue construido para x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
Licencia GPLv3+: GNU GPL versión 3 o posterior <http://gnu.org/licenses/gpl.html>
Este es software libre: cualquiera es libre para redistribuirlo y modificarlo.
No existe GARANTÍA ALGUNA, hasta los límites permitidos por las leyes aplicables.
```

5.2. Instalando dependencias

Para la instalación de las dependencias requeridas, se deben ejecutar los siguientes comandos:

```
sudo apt update
sudo apt install -y build-essential linux-headers-$(uname -r) gcc make
```

6. Ejemplos de drivers propuestos por la cátedra

6.1. Módulo básico (drv1)

El módulo básico `drv1` representa la implementación más simple posible. Su propósito fundamental es ilustrar el ciclo de vida de un módulo del *kernel*, el cual abarca exclusivamente sus etapas de carga (mediante el comando `insmod`) y descarga (a través de `rmmmod`).

En el marco de este desarrollo, se introducen los siguientes conceptos clave:

- `module_init()` / `module_exit()`: Macros empleadas para registrar las rutinas de inicialización (constructor) y limpieza (destructor) del módulo.
- `printk()`: Función equivalente a `printf`, pero diseñada para operar en el espacio del *kernel*, encargada de escribir mensajes en el registro (*log*) del sistema.
- `MODULE_LICENSE("GPL")`: Declaración de uso obligatorio cuya finalidad es evitar que el *kernel* adquiera el estado de *tainted* (contaminado).

A continuación se muestra la prueba hecha sobre el driver:

```
+ FuentesDrv git:(main) X make DRV=1
mkdir -p /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv/build
make -C /lib/modules/6.14.0-37-generic/build M=/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.14.0-37-generic'
make[2]: se entra en el directorio '/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0
  You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
  CC [M]   Drv1.o
  MODPOST Module.symvers
  CC [M]   Drv1.mod.o
  CC [M]   .module-common.o
  LD [M]   Drv1.ko
  BTF [M]  Drv1.ko
Skipping BTF generation for Drv1.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv'
make[1]: se sale del directorio '/usr/src/linux-headers-6.14.0-37-generic'
find /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv -maxdepth 1 -name "*.o" -o -name "*.ko" -o -name
ame "*.symvers" -o -name "*.cmd" | xargs -I{} mv -f {} /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv
find /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv -maxdepth 1 -name "*.cmd" -o -name "*.o" | xarg
on/TP5/FuentesDrv/build/2>/dev/null || true
+ FuentesDrv git:(main) X make DRV=1 load
sudo insmod /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv/build/Drv1.ko
[sudo] contraseña para esteban:
+ FuentesDrv git:(main) X lsmod | grep Drv1
Drv1                12288  0
```

```
+ FuentesDrv git:(main) X make DRV=1 log
sudo dmesg | tail -10
[ 15.285870] 02:48:22.997727 X11 events received X11 event (89)
[ 15.286068] 02:48:22.997892 X11 events RRSscreenChangeNotify event received
[ 15.287101] 02:48:22.998882 X11 events Monitor 0 (w,h)=(1920,1080) (x,y)=(0,0)
[ 15.287445] 02:48:22.999133 X11 events Sending monitor positions (8 of them) to the h
[ 365.324367] hrtimer: interrupt took 2131427 ns
[ 719.802222] kauditd_printk_skb: 121 callbacks suppressed
[ 719.802226] audit: type=1400 audit(1780369208.994:133): apparmor="DENIED" operation="cap
apability=12 capname="net_admin"
[ 1956.271946] Drv1: loading out-of-tree module taints kernel.
[ 1956.271954] Drv1: module verification failed: signature and/or required key missing - t
[ 1956.273573] SdeC: drv1 Registrado exitosamente...!!
+ FuentesDrv git:(main) X make DRV=1 unload
sudo rmmod Drv1
+ FuentesDrv git:(main) X make DRV=1 log
sudo dmesg | tail -10
[ 15.286068] 02:48:22.997892 X11 events RRSscreenChangeNotify event received
[ 15.287101] 02:48:22.998882 X11 events Monitor 0 (w,h)=(1920,1080) (x,y)=(0,0)
[ 15.287445] 02:48:22.999133 X11 events Sending monitor positions (8 of them) to the h
[ 365.324367] hrtimer: interrupt took 2131427 ns
[ 719.802222] kauditd_printk_skb: 121 callbacks suppressed
[ 719.802226] audit: type=1400 audit(1780369208.994:133): apparmor="DENIED" operation="cap
apability=12 capname="net_admin"
[ 1956.271946] Drv1: loading out-of-tree module taints kernel.
[ 1956.271954] Drv1: module verification failed: signature and/or required key missing - t
[ 1956.273573] SdeC: drv1 Registrado exitosamente...!!
[ 2203.857787] SdeC: drv1 dice Adios mundo cruel...!!
+ FuentesDrv git:(main) X
```

6.2. Registro de major/minor (drv2)

El módulo `drv2` incorpora el registro dinámico de números de dispositivo mediante la función `alloc_chrdev_region()`. A continuación, se detallan los conceptos clave introducidos en esta etapa:

- `alloc_chrdev_region()`: Función mediante la cual el *kernel* asigna automáticamente un número *major* que se encuentre libre en el sistema.
- `unregister_chrdev_region()`: Rutina encargada de liberar los rangos de números asignados al momento de descargar el módulo, garantizando la devolución de recursos.
- **Nodos de dispositivo**: Los números *minor* asignados se vuelven operativos en el espacio de usuario al crear los archivos de dispositivo correspondientes de forma manual mediante el comando `mknod`.

A continuación se muestra la prueba hecha sobre el driver:

```
→ FuentesDrv git:(main) X make DRV=2
mkdir -p /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv/build
make -C /lib/modules/6.14.0-37-generic/build M=/home/esteban/Sistemas-de-Computacion/TP5/Fu
make[1]: se entra en el directorio '/usr/src/linux-headers-6.14.0-37-generic'
make[2]: se entra en el directorio '/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
You are using:
gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] Drv2.o
MODPOST Module.symvers
CC [M] Drv2.mod.o
CC [M] .module-common.o
LD [M] Drv2.ko
BTF [M] Drv2.ko
Skipping BTF generation for Drv2.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv'
make[1]: se sale del directorio '/usr/src/linux-headers-6.14.0-37-generic'
find /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv -maxdepth 1 -name "*.o" -o -name
ame "*.symvers" -o -name "*.cmd" | xargs -I{} mv -f {} /home/esteban/Sistemas-de-Computac
find /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv -maxdepth 1 -name "*.cmd" -o -n
on/TP5/FuentesDrv/build/ 2>/dev/null || true
→ FuentesDrv git:(main) X make DRV=2 load
sudo insmod /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv/build/Drv2.ko
→ FuentesDrv git:(main) X cat /proc/devices | grep SdeC
240 SdeC_Driver2
```

```
→ FuentesDrv git:(main) X cat /proc/devices | grep SdeC
240 SdeC_Driver2
→ FuentesDrv git:(main) X make DRV=2 log
sudo dmesg | tail -10
[ 15.287445] 02:48:22.999133 X11 events Sending monitor positions (8 of them) to the host: VINI_SUCCESS
[ 365.324367] hrtimer: interrupt took 2131427 ns
[ 719.802222] kauditd_printk_skb: 121 callbacks suppressed
[ 719.802226] audit: type=1400 audit(1780369208.994:133): apparmor="DENIED" operation="capable" class="cap
apability=12 capname="net_admin"
[ 1956.271946] Drv1: loading out-of-tree module taints kernel.
[ 1956.271954] Drv1: module verification failed: signature and/or required key missing - tainting kernel
[ 1956.273573] SdeC: drv1 Registrado exitosamente...!!
[ 2203.857787] SdeC: drv1 dice Adios mundo cruel...!!
[ 2870.387523] SdeC_drv2 Registrado exitosamente...!!
[ 2870.387527] <Major, Minor>: <240, 0>
→ FuentesDrv git:(main) X make DRV=2 unload
sudo rmmod Drv2
→ FuentesDrv git:(main) X
→ FuentesDrv git:(main) X make DRV=2 log
sudo dmesg | tail -10
[ 365.324367] hrtimer: interrupt took 2131427 ns
[ 719.802222] kauditd_printk_skb: 121 callbacks suppressed
[ 719.802226] audit: type=1400 audit(1780369208.994:133): apparmor="DENIED" operation="capable" class="cap
apability=12 capname="net_admin"
[ 1956.271946] Drv1: loading out-of-tree module taints kernel.
[ 1956.271954] Drv1: module verification failed: signature and/or required key missing - tainting kernel
[ 1956.273573] SdeC: drv1 Registrado exitosamente...!!
[ 2203.857787] SdeC: drv1 dice Adios mundo cruel...!!
[ 2870.387523] SdeC_drv2 Registrado exitosamente...!!
[ 2870.387527] <Major, Minor>: <240, 0>
[ 3003.800498] SdeC_drv2 dice Adios mundo cruel...!!
```

6.3. CDD completo usando /dev (drv3)

El módulo `drv3` implementa la creación automática del archivo `/dev/SdeC_drv3` a través de `udev`. Conceptos clave introducidos:

- `class_create()`: Crea una clase de dispositivo en `/sys/class/`.
- `device_create()`: Genera la entrada en `/sys/class/` para que `udev` cree el nodo en `/dev/SdeC_drv3`.
- `cdev_init()` + `cdev_add()`: Inicializan y registran el dispositivo de caracteres (*struct cdev*) en el *kernel*.
- `file_operations`: Estructura que vincula las llamadas de sistema (`open`, `read`, `write`, `release`) con las funciones del *driver*.

A continuación se muestra la prueba hecha sobre el driver:

```
+ FuentesDrv git:(main) * make DRV=3
mkdir -p /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv/build
make -C /lib/modules/6.14.0-37-generic/build M=/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.14.0-37-generic'
make[2]: se entra en el directorio '/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] Drv3.o
MODPOST Module.symvers
CC [M] Drv3.mod.o
CC [M] .module-common.o
LD [M] Drv3.ko
BTF [M] Drv3.ko
Skipping BTF generation for Drv3.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv'
make[1]: se sale del directorio '/usr/src/linux-headers-6.14.0-37-generic'
find /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv -maxdepth 1 -name "*.o" -o -name "*.ko" -o -name
ame "*.symvers" -o -name "*.cmd" | xargs -I{} mv -f {} /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv
find /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv -maxdepth 1 -name "*.cmd" -o -name "*.o" | xarg
on/TP5/FuentesDrv/build/ 2>/dev/null || true
+ FuentesDrv git:(main) * make DRV=3 load
sudo insmod /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv/build/Drv3.ko
[sudo] contraseña para esteban:
+ FuentesDrv git:(main) * ls -la /dev/SdeC_drv3
crw----- 1 root root 240, 0 jun  2 00:56 /dev/SdeC_drv3
+ FuentesDrv git:(main) * sudo cat /dev/SdeC_drv3
+ FuentesDrv git:(main) * echo "hola" | sudo tee /dev/SdeC_drv3
hola
```

```
+ FuentesDrv git:(main) * make DRV=3 log
sudo dmesg | tail -10
[ 2870.387523] SdeC_drv2 Registrado exitosamente...!!
[ 2870.387527] <Major, Minor>: <240, 0>
[ 3003.800498] SdeC_drv2 dice Adios mundo cruel...!!
[ 4125.432888] SdeC_drv3 Registrado exitosamente...!!
[ 4171.981552] Driver3_SdeC: open()
[ 4171.981569] Driver3_SdeC: read()
[ 4171.981580] Driver3_SdeC: close()
[ 4195.569371] Driver3_SdeC: open()
[ 4195.569416] Driver3_SdeC: write()
[ 4195.569418] Driver3_SdeC: close()
+ FuentesDrv git:(main) * make DRV=3 unload
sudo rmmod Drv3
+ FuentesDrv git:(main) * make DRV=3 log
sudo dmesg | tail -10
[ 2870.387527] <Major, Minor>: <240, 0>
[ 3003.800498] SdeC_drv2 dice Adios mundo cruel...!!
[ 4125.432888] SdeC_drv3 Registrado exitosamente...!!
[ 4171.981552] Driver3_SdeC: open()
[ 4171.981569] Driver3_SdeC: read()
[ 4171.981580] Driver3_SdeC: close()
[ 4195.569371] Driver3_SdeC: open()
[ 4195.569416] Driver3_SdeC: write()
[ 4195.569418] Driver3_SdeC: close()
[ 4225.498353] SdeC_drv3 dice Adios mundo cruel...!!
```

6.4. R/W con espacio de usuario (drv4)

El módulo drv4 extiende a drv3 implementando la transferencia real de datos entre el *kernel* y el espacio de usuario.

Conceptos clave introducidos:

- `copy_to_user()`: Copia datos de forma segura del *kernel* al usuario (operación `read`).
- `copy_from_user()`: Copia datos de forma segura del usuario al *kernel* (operación `write`).
- `*off` (offset): Controla la posición de lectura para indicar el fin de archivo (*EOF*).
- **Almacenamiento**: El *driver* gestiona un único byte en una variable global `c`.
- **Aislamiento de memoria**: No se puede acceder directamente a los punteros de usuario porque ambos espacios de memoria están estrictamente separados y protegidos; estas funciones garantizan una transferencia segura y validada.

A continuación se muestra la prueba hecha sobre el driver:

```
+ FuentesDrv git:(main) X make DRV=4
mkdir -p /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv/build
make -C /lib/modules/6.14.0-37-generic/build M=/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.14.0-37-generic'
make[2]: se entra en el directorio '/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
CC [M] Drv4.o
MODPOST Module.symvers
CC [M] Drv4.mod.o
CC [M] .module-common.o
LD [M] Drv4.ko
BTF [M] Drv4.ko
Skipping BTF generation for Drv4.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv'
make[1]: se sale del directorio '/usr/src/linux-headers-6.14.0-37-generic'
find /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv -maxdepth 1 -name "*.o" -o -name "*.ko" -o -name
ame "*.symvers" -o -name "*.cmd" | xargs -I{} mv -f {} /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv
find /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv -maxdepth 1 -name "*.cmd" -o -name "*.o" | xarg
on/TP5/FuentesDrv/build/ 2>/dev/null || true
+ FuentesDrv git:(main) X make DRV=4 load
sudo insmod /home/esteban/Sistemas-de-Computacion/TP5/FuentesDrv/build/Drv4.ko
+ FuentesDrv git:(main) X ls -la /dev/SdeC_drv4
crw----- 1 root root 240, 0 jun  2 01:06 /dev/SdeC_drv4
+ FuentesDrv git:(main) X echo -n "X" | sudo tee /dev/SdeC_drv4
X
+ FuentesDrv git:(main) X sudo cat /dev/SdeC_drv4
X
```

```
+ FuentesDrv git:(main) X make DRV=4 log
sudo dmesg | tail -10
[ 4195.569418] Driver3_SdeC: close()
[ 4225.498353] SdeC_drv3 dice Adios mundo cruel...!!
[ 4682.841376] SdeC_drv4: Registrado exitosamente...!!
[ 4766.795244] SdeC_drv4: open()
[ 4766.795319] SdeC_drv4: write()
[ 4766.795323] SdeC_drv4: close()
[ 4788.256113] SdeC_drv4: open()
[ 4788.256137] SdeC_drv4: read()
[ 4788.256161] SdeC_drv4: read()
[ 4788.256175] SdeC_drv4: close()
+ FuentesDrv git:(main) X make DRV=4 unload
sudo rmmod Drv4
+ FuentesDrv git:(main) X make DRV=4 log
sudo dmesg | tail -10
[ 4225.498353] SdeC_drv3 dice Adios mundo cruel...!!
[ 4682.841376] SdeC_drv4: Registrado exitosamente...!!
[ 4766.795244] SdeC_drv4: open()
[ 4766.795319] SdeC_drv4: write()
[ 4766.795323] SdeC_drv4: close()
[ 4788.256113] SdeC_drv4: open()
[ 4788.256137] SdeC_drv4: read()
[ 4788.256161] SdeC_drv4: read()
[ 4788.256175] SdeC_drv4: close()
[ 4827.201070] SdeC_drv4: dice Adios mundo cruel...!!
```

7. Generación de señales

Se desarrolló un *script* en *Python* para generar dos señales mediante los puertos GPIO de una *Raspberry Pi*:

- **Señal senoidal:** Asignada al puerto GPIO18 (Período: 1 s).
- **Señal cuadrada:** Asignada al puerto GPIO16 (Período: 1 s).
- **Reconstrucción de señal:** Dado que los puertos GPIO son estrictamente digitales (valores 0 ó 1), se implementa una técnica de modulación que promedia N bits consecutivos para reconstruir el valor analógico original de las señales.

7.1. Arquitectura usada para la comunicación

La arquitectura de comunicación opera bajo un modelo cliente-servidor a través de un *socket* de dominio Unix:

- **Servidor (*Script*):** Instancia el *socket* en `/tmp/tmp-gpio.sock` y aguarda conexiones entrantes.
- **Cliente (QEMU):** Al ejecutar `run.sh`, el emulador se inicializa y se conecta al *socket* disponible.
- **Interacción:** El *script* transmite comandos mediante el protocolo *qtest* de QEMU, realizando escrituras directas sobre los registros de memoria del controlador GPIO (chip BCM2837 de la *Raspberry Pi 3*).

7.2. Señal PDM

La codificación de las señales se define bajo los siguientes criterios:

- **Señal senoidal (PDM):** Utiliza modulación por densidad de pulsos (*Pulse Density Modulation*). La información analógica se representa según la cantidad de unos (1) presentes en una ventana de tiempo determinada, y no por el ancho del pulso.
- **Señal cuadrada:** Su generación consiste en una conmutación directa (*toggle*) entre 0 y 1 con un período de 1 s.

Aclaración importante

Las restricciones del entorno de emulación imponen las siguientes condiciones técnicas:

- **Fidelidad de la señal:** La representación precisa de la onda senoidal requiere una alta densidad de muestras por período.
- **Limitación de QEMU:** Debido a la capacidad de cómputo del emulador, el límite máximo operativo es de 400 muestras.
- **Restricción de período:** Para mantener una resolución aceptable bajo esta limitación, el período mínimo de la señal senoidal se fija en 4 s.

7.3. Orden de arranque

La secuencia operativa se resume en los siguientes pasos:

1. `python3 signals_rpi.py`: Genera el *socket* y espera la conexión de forma bloqueante (Terminal 1).
2. `./run.sh`: Inicializa QEMU y lo conecta automáticamente al *socket* (Terminal 2).
3. **Transmisión**: Las señales comienzan a fluir de manera autónoma.

Los estados lógicos enviados se imprimen en tiempo real en la terminal 1, tal como se ilustra a continuación:

t=39.050s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.100s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.150s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.200s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.250s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.300s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.350s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.400s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.450s	GPIO18 (PDM)=0	GPIO16 (CUA)=1
t=39.500s	GPIO18 (PDM)=1	GPIO16 (CUA)=1
t=39.550s	GPIO18 (PDM)=0	GPIO16 (CUA)=0
t=39.600s	GPIO18 (PDM)=0	GPIO16 (CUA)=0
t=39.650s	GPIO18 (PDM)=0	GPIO16 (CUA)=0
t=39.700s	GPIO18 (PDM)=0	GPIO16 (CUA)=0
t=39.750s	GPIO18 (PDM)=0	GPIO16 (CUA)=0

8. CDD

El driver desarrollado conecta la aplicación de usuario con el kernel mediante las siguientes funciones:

- **Interfaz:** Expone un dispositivo de caracteres en `/dev`.
- **Control:** Permite seleccionar cuál de las dos señales GPIO leer.
- **Salida:** Devuelve el valor lógico leído en formato de texto plano.

8.1. Flujo del sistema

A continuación se muestra el diagrama de flujo del sistema:

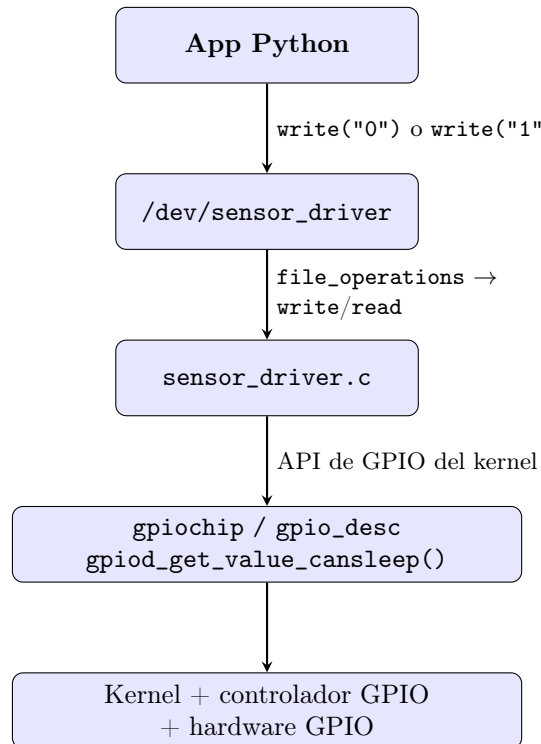


Figura 2: Diagrama de flujo de datos desde la aplicación al hardware

Entonces si queremos iniciar el sistema, debemos ejecutar los siguientes comandos:

```
+ gemu-gpio-rpi git:(main) X sudo ./gemugpio-rpi
[ ] Virtual GPIO manager
[ ] Listening for connections
(gpio)> set 16 1
b'OK\r\n'
(gpio)> set 16 0
b'OK\r\n'
(gpio)> set 18 1
b'OK\r\n'
(gpio)> set 18 0
b'OK\r\n'
(gpio)>
```

La imagen muestra una terminal con el programa 'gemu-gpio-rpi' ejecutándose. El programa está en modo de escucha y recibe comandos de configuración de pines GPIO. Los comandos mostrados son 'set 16 1', 'set 16 0', 'set 18 1' y 'set 18 0', cada uno respondiendo con 'b'OK\r\n'.

```
* qemu-setup git:(main) X sudo ./run.sh
[ ] root image: /tmp/raspbios_lite_armhf_latest.img
[I 0.000000] OPENED
usbnet: failed control transaction: request 0x8006 value 0x600 index 0x0 length 0xa
usbnet: failed control transaction: request 0x8006 value 0x600 index 0x0 length 0xa
usbnet: failed control transaction: request 0x8006 value 0x600 index 0x0 length 0xa
[R +9.520549] writel 0x3f20001c 0x10000
[S +9.520620] OK
[R +12.476063] writel 0x3f200028 0x10000
[S +12.476086] OK
```

```
* qemu-gpio-rpi git:(main) X ssh -X -p 50022 esteban@127.0.0.1
esteban@127.0.0.1's password:
Linux raspberrypi 6.8.50+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.8.50-1+rpt1 (2026-05-15) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Jun 3 15:30:10 2026 from 10.0.2.2
```

```
esteban@raspberrypi:~ $ cd ~/TP5/CDD
make clean
make load
rm -rf "/home/esteban/TP5/CDD/build"
make -C "/lib/modules/6.8.50+rpt-rpi-v8/build" M="/home/esteban/TP5/CDD" clean
make[1]: Entering directory '/usr/src/linux-headers-6.8.50+rpt-rpi-v8'
make[1]: Leaving directory '/usr/src/linux-headers-6.8.50+rpt-rpi-v8'
mkdir -p "/home/esteban/TP5/CDD/build"
make -C "/lib/modules/6.8.50+rpt-rpi-v8/build" M="/home/esteban/TP5/CDD" modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.50+rpt-rpi-v8'
CC [M] /home/esteban/TP5/CDD/sensor_driver.o
MODPOST /home/esteban/TP5/CDD/Module.symvers
CC [M] /home/esteban/TP5/CDD/sensor_driver.mod.o
CC [M] /home/esteban/TP5/CDD/.module-common.o
LD [M] /home/esteban/TP5/CDD/sensor_driver.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.8.50+rpt-rpi-v8'
find "/home/esteban/TP5/CDD" -maxdepth 1 \! ( -name "*.o" -o -name "*.ko" -o -name "*.mod*" -o -name "*.order" -o -name "*.symvers" -o -name "*.cmd" ) -exec mv -f {} "/home/esteban/TP5/CDD/build/" \; 2>/dev/null || true
find "/home/esteban/TP5/CDD" -maxdepth 1 -name "*.cmd" -exec mv -f {} "/home/esteban/TP5/CDD/build/" \; 2>/dev/null || true
sudo rmmod sensor_driver 2>/dev/null || true
[sudo] password for esteban:
sudo insmod "/home/esteban/TP5/CDD/build/sensor_driver.ko"
sudo chmod 666 /dev/sensor_driver
```

8.2. Inicialización y Lectura de Descriptores GPIO

Al momento de inicializar el módulo, el controlador (*driver*) localiza el `gpio_device` disponible y obtiene su `gpio_chip` asociado. Posteriormente, efectúa la reserva de dos descriptores GPIO mediante la API de descriptores del kernel de Linux:

- **GPIO16** asignado como la señal 0.
- **GPIO18** asignado como la señal 1.

Cabe destacar que la lectura de los pines no se realiza a través de la interfaz `sysfs`, sino que se implementa utilizando la función `gpiod_get_value_cansleep()`. Esta rutina permite consultar el estado lógico del descriptor GPIO de manera segura, respetando las restricciones del kernel y posibilitando su ejecución en contextos donde la operación de lectura es susceptible de entrar en estado de suspensión (*sleep*).

Finalmente, cuando la aplicación en el espacio de usuario efectúa una lectura sobre el nodo `/dev/sensor_driver`, el módulo retorna una cadena de texto indicando los estados actuales, con el siguiente formato:

```
GPIO16:0
GPIO18:1
```

8.3. Registro del Dispositivo y Operaciones de Archivo

El módulo registra un dispositivo de caracteres (*character device*) mediante el uso de las funciones `alloc_chrdev_region()`, `class_create()`, `device_create()` y `cdev_add()`. Esta secuencia de inicialización instruye al kernel para generar el nodo correspondiente en el directorio `/dev`, enlazando las operaciones de archivo (*file operations*) del módulo con la aplicación en el espacio de usuario.

Las operaciones implementadas por el driver son las siguientes:

- `open()` y `release()`: encargadas de gestionar el ciclo de acceso al dispositivo.
- `write()`: utilizada para seleccionar dinámicamente el pin GPIO que se desea monitorear.
- `read()`: encargada de consultar el estado lógico actual del pin elegido.

Específicamente, la función `write()` procesa un byte de entrada interpretado de la siguiente manera:

- Un valor de 0 selecciona el **GPIO16**.
- Un valor de 1 selecciona el **GPIO18**.

Posteriormente, la función `read()` formatea la respuesta en una cadena de texto y la transfiere de manera segura al espacio de usuario mediante `copy_to_user()`. De esta manera, se garantiza que la aplicación no acceda directamente al hardware subyacente, sino que interactúe exclusivamente a través de la interfaz abstracta definida por el *driver*.

8.4. Justificación Arquitectónica

El uso de la API moderna de descriptores GPIO (`gpiolib`), en lugar de interfaces depreciadas como `sysfs`, provee al controlador de una interfaz más robusta y segura. Este enfoque centrado en descriptores (`gpiod`) garantiza una integración óptima con la infraestructura actual del kernel de Linux.

8.5. Validación Funcional

La verificación del *driver* consistió en cargar el módulo y confirmar la creación del nodo `/dev/sensor_driver`. Posteriormente, mediante la aplicación de usuario, se validó la correcta selección del GPIO objetivo a través de operaciones de escritura, y se corroboró que las lecturas retornaran los estados lógicos esperados, reflejándose correctamente en el gráfico.

9. Aplicación de usuario en Python

La aplicación en el espacio de usuario, desarrollada en Python, utiliza `tkinter` para la interfaz gráfica y `matplotlib` para la representación en tiempo real. Su función es configurar el GPIO activo escribiendo en `/dev/sensor_driver`, para luego muestrear su estado lógico a una frecuencia de 1 Hz y graficar su evolución temporal.

9.1. Flujo de Ejecución de la Aplicación

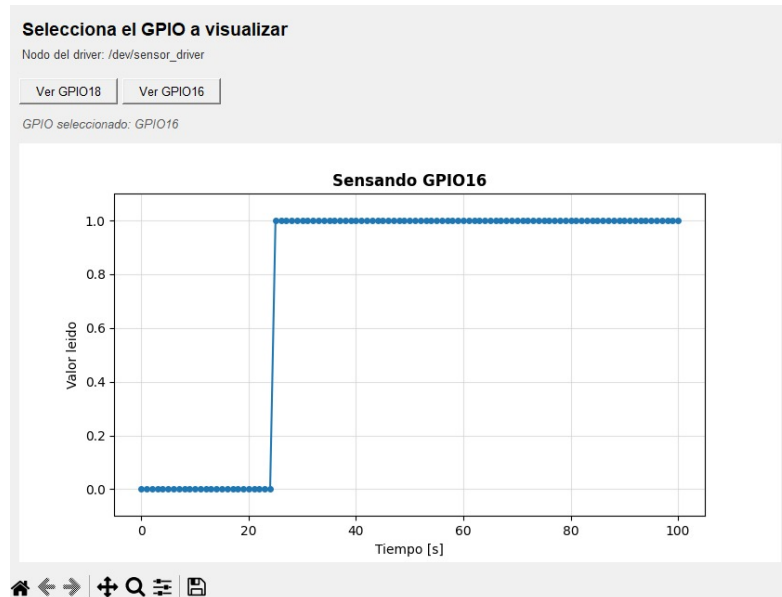
La secuencia de operación del sistema es la siguiente:

1. El usuario selecciona el **GPIO16** o el **GPIO18** en la interfaz.
2. La aplicación escribe un 0 o un 1 en `/dev/sensor_driver` para actualizar el pin monitoreado por el *driver*.
3. Un hilo de lectura en segundo plano consulta el nodo del dispositivo una vez por segundo.
4. La respuesta textual se procesa mediante una expresión regular y el valor extraído actualiza dinámicamente el gráfico de `matplotlib`.

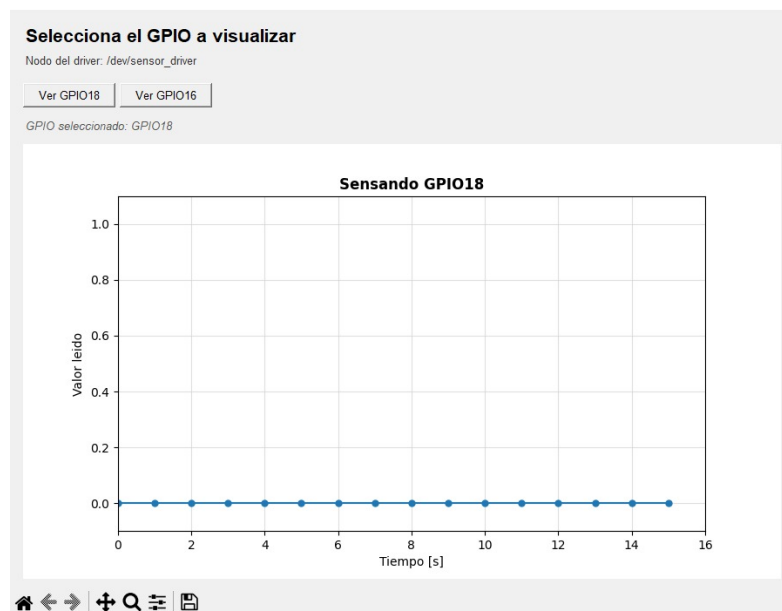
A modo de prueba, se muestra a continuación la inicialización de la aplicación ejecutada desde la propia Raspberry Pi simulada en QEMU:

```
esteban@raspberrypi:~/TP5 $ python3 app.py
[app] Selector cambiado a GPIO16
[app] Cambio de senal detectado: GPIO16. Grafico reiniciado
```

A continuación se observa la conmutación de la señal en la interfaz al alternar entre el **GPIO16** y el **GPIO18**. Inicialmente, se selecciona el **GPIO16**:



Luego se selecciona el **GPIO18**:



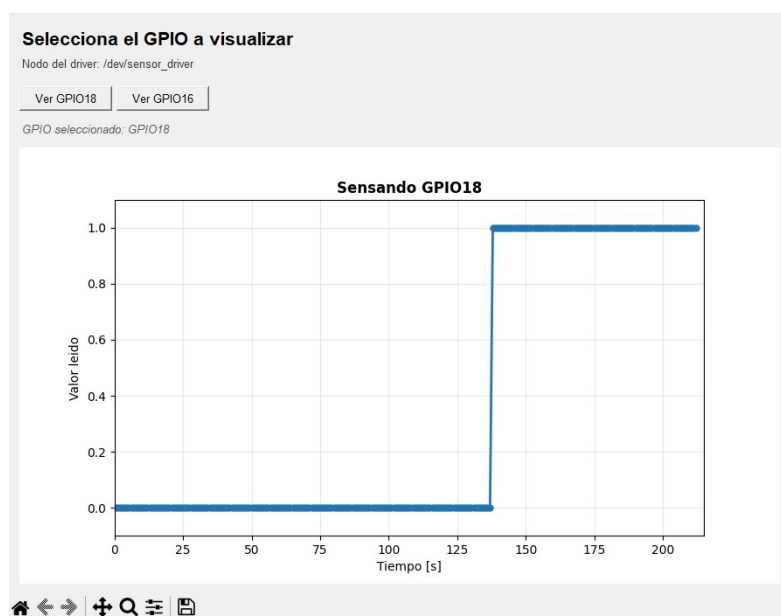
Y si ahora miramos el estado de la consola de la RaspberryPi:

```
esteban@raspberrypi:~/TP5 $ python3 app.py
[app] Selector cambiado a GPIO16
[app] Cambio de senal detectado: GPIO16. Grafico reiniciado
[app] Selector cambiado a GPIO18
[app] Cambio de senal detectado: GPIO18. Grafico reiniciado
```

Si ponemos a 1 el **GPIO18** desde el emulador:

```
→ gemu-gpio-rpi git:(main) ✕ ./gemugpio-rpi
[ ] Virtual GPIO manager
[ ] Listening for connections
(gpio)> set 16 1
b'OK\r\n'
(gpio)> set 16 0
b'OK\r\n'
(gpio)> set 18 1
b'OK\r\n'
(gpio)> %
```

Vemos como se actualiza también el gráfico:



9.2. Detalles de Implementación

Para evitar el bloqueo de la interfaz gráfica, la aplicación delega la adquisición de datos a un hilo de ejecución en segundo plano. Este hilo aguarda la configuración de un pin, abre el dispositivo en modo lectura y procesa la cadena de texto retornada mediante el patrón **GPIO<numero>:<valor>**. Es la propia aplicación la que asume la responsabilidad de la conversión a tipos numéricos y el escalado de la señal, liberando al kernel de cualquier tarea relacionada con el formato visual.

Conforme a las restricciones del proyecto, la representación gráfica invierte la disposición tradicional de los ejes: el valor lógico de la señal se proyecta sobre el eje de abscisas (X), mientras que el tiempo transcurrido ocupa el eje de ordenadas (Y).

Por su parte, el hilo principal gestiona los eventos de la ventana y el ciclo de actualización de la interfaz. Al conmutar de GPIO, el sistema purga las muestras previas, reinicia la métrica temporal y actualiza el título del gráfico para reflejar exclusivamente la lectura activa.

9.3. Acoplamiento con el Driver

El software de usuario opera con total abstracción de la arquitectura interna del kernel y del hardware subyacente. Su única dependencia radica en el contrato de interfaz definido por el nodo del dispositivo:

- Escribir 0 o 1 para seleccionar el pin objetivo.
- Leer periódicamente (a una frecuencia de 1 Hz) el estado lógico actual.

Esta estricta separación de responsabilidades garantiza que la aplicación sea completamente independiente de los mecanismos de bajo nivel que emplea el *driver* para interactuar con los registros físicos.

10. Prueba final

A modo de cierre del apartado práctico, se llevó a cabo una prueba de integración que articuló la generación de señales, el *Character Device Driver* (CDD) y la aplicación en el espacio de usuario, permitiendo validar el correcto funcionamiento global del sistema.

10.1. Terminal 1: Generación de señales

```
python3 signals_rpi.py
```

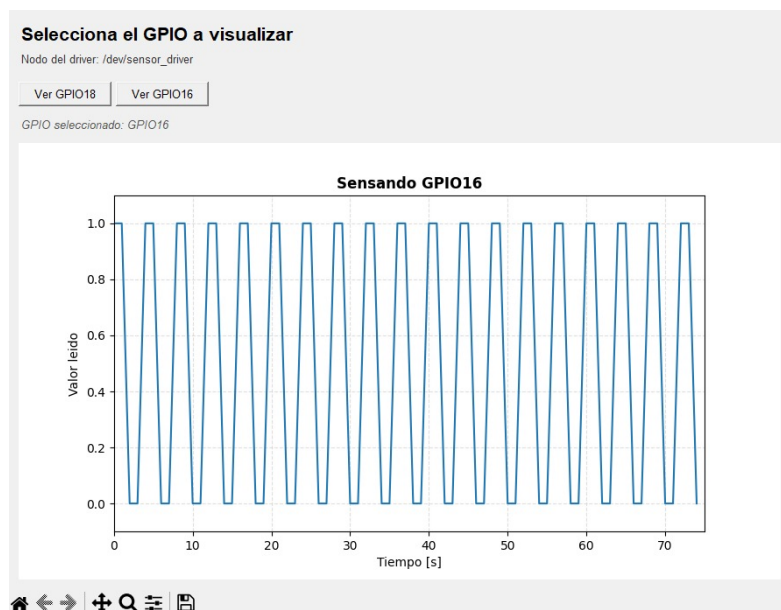
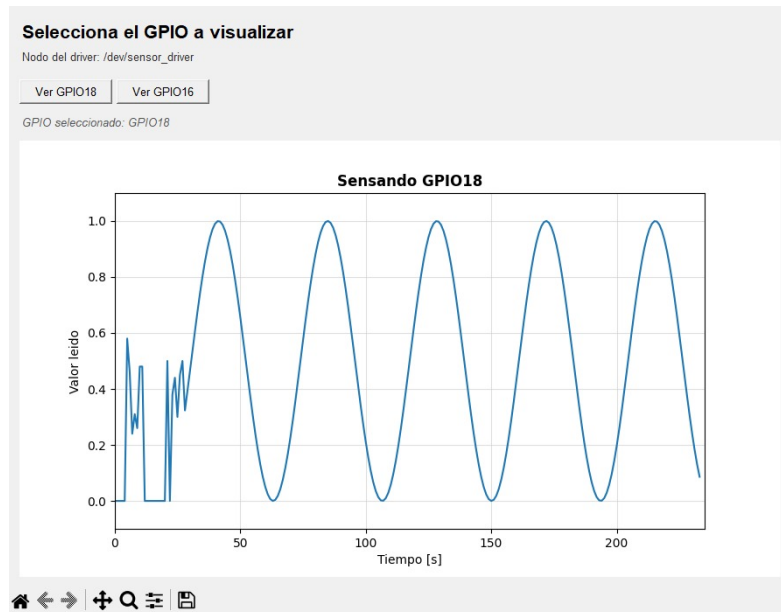
10.2. Terminal 2: QEMU

```
./run.sh
```

10.3. Terminal 3: SSH en la Raspberry Pi simulada

```
ssh -X -p 50022 esteban@127.0.0.1
cd ~/TP5/CDD
make load
cd ~/TP5
python3 app.py
```

Por último, a continuación se ilustra el proceso de generación de señales, detallando de manera independiente las formas de onda cuadrada y senoidal:



11. Conclusión

El desarrollo de este trabajo práctico posibilitó la integración efectiva entre el espacio de usuario y el espacio de kernel mediante la implementación de un *Character Device Driver* (CDD). Se logró una estricta separación de responsabilidades: el controlador asume de manera exclusiva el acceso y gestión del hardware GPIO, mientras que la aplicación se restringe a la representación gráfica y la interacción con el usuario.

Una de las deducciones fundamentales es que el acceso directo a los pines físicos desde aplicaciones de alto nivel constituye una práctica desaconsejada en sistemas operativos modernos. El diseño óptimo delega esta tarea a un módulo del kernel, exponiendo el dispositivo a través de la abstracción de un archivo estándar. Este paradigma simplifica la lógica de la aplicación de usuario, maximiza el aislamiento de los procesos y aprovecha la infraestructura nativa de Linux.

Asimismo, se corroboró la superioridad de la API moderna de descriptores GPIO (`gpiolib`) frente a interfaces depreciadas como `sysfs`. Su adopción resulta en un driver robusto, alineado con las convenciones actuales de desarrollo del kernel de Linux y con un alto grado de mantenibilidad.

En definitiva, la solución implementada evidencia de manera exitosa una arquitectura segmentada en tres capas fundamentales:

- El **hardware** subyacente (en este caso, provisto por el entorno de emulación).
- El ***driver*** que opera como capa de abstracción y control.
- La **aplicación de usuario** que adquiere, procesa y visualiza gráficamente la información resultante.